

(请大家预习)

第6章第1讲

人工神经网络

Artificial Neural Networks

张 燕 明

ymzhang@nlpr.ia.ac.cn

peopleucas.ac.cn/~ymzhang

模式分析与学习课题组(PAL)

多模态人工智能系统实验室 中科院自动化所

助教： 杨 奇 (yangqi2021@ia.ac.cn)

张 涛 (zhangtao2021@ia.ac.cn)

第6讲内容回顾

- 生成模型

$$P(\omega_i | \mathbf{x}) = \frac{p(\mathbf{x} | \omega_i)P(\omega_i)}{p(\mathbf{x})}$$

- 对类条件概率密度 $p(\mathbf{x}|\omega_i)$ 的估计通常很困难

- 判别模型

- 估计类后验概率或判别函数，直接对样本分类

- 两类问题：

- 只需估计一个判别函数 $g(\mathbf{x})$
- **决策准则：** $g(\mathbf{x}) > 0$, 则预测 $\mathbf{x}$ 属于正类，否则为负类

- c 类问题：

- 把一个多类问题转化为多个两类问题，需估计多个判别函数。具体方法：**一对多**、**一对一**、**多对多**等

第6讲内容回顾

- 判别模型

- 一对多方法

- 对每类训练一个判别函数 $g_i(\mathbf{x})$ ，用于区分 ω_i 类和其他类
 - 决策准则(最大判别函数准则):

如果 $g_i(\mathbf{x}) > g_j(\mathbf{x}), \forall j \neq i$ ，则 $\mathbf{x}$ 被分为 ω_i 类

- 线性判别函数 $g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$

- 决策面 $g(\mathbf{x})=0$ 是一个超平面，将特征空间分为两部分
 - 广义线性判别函数：通过非线性特征变换将 $\mathbf{x}$ 映射到新特征空间，在新空间使用线性判别函数

第6讲内容回顾

- 线性判别函数的学习

- 学习任务：估计 $\mathbf{a}$
- 学习准则

- **感知器准则** $J_p(\mathbf{a}) = \sum_{\mathbf{y} \in Y} (-\mathbf{a}^T \mathbf{y})$, Y 为 $\mathbf{a}^T \mathbf{y} \leq 0$ 的样本集合

- **松弛准则** $J_r(\mathbf{a}) = \frac{1}{2} \sum_{\mathbf{y} \in Y} \frac{(\mathbf{a}^T \mathbf{y} - b)^2}{\|\mathbf{y}\|^2}$, Y 为 $\mathbf{a}^T \mathbf{y} \leq b$ 的样本集合

- **平方误差准则** $J_s(\mathbf{a}) = \sum_{i=1}^n (\mathbf{a}^T \mathbf{y}_i - b_i)^2$

- **极大似然准则** $\sum_{i=1}^n \ln P(\omega_i | y_i)$ ω_i 表示 y_i 的类别

- 优化算法：

- **梯度下降**：（批量、单样本），（可变增量、固定增量）

优点：简单；缺点：太简单

内容提要

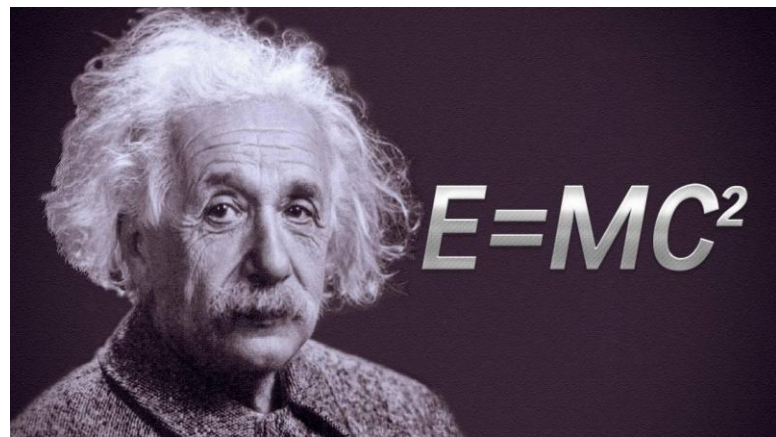
- 介绍
 - 发展历史
- 基本模型
 - 单层感知器、多层感知器、RBF网络
- 扩展模型
 - Hopfield网络、RBM、CNN、Autoencoder、RNN、LSTM、Transformer、GNN等
- 模型的学习
 - BP算法等

第一节

人工神经网络发展历程

6.1.1 概述

- 人脑



人是地球上具有最高智慧的动物，而人的智慧均来自大脑，人类靠大脑进行思考、想象、记忆和推理判断等。

6.1.1 概述

- 人脑

长期以来，科学家一直致力于人脑内部结构和功能的探讨和研究，并试图建立模仿人类大脑的计算机。

虽然到目前对大脑的内部工作机理还不完全清楚，但对其结构已有所了解。

6.1.1 概述

- 人脑

大脑里有约800亿个神经细胞（神经元）。

每个神经元可看作是一个小的处理单元，这些神经元按某种方式连接起来，形成大脑内部的生物神经元网络。

神经网络中各神经元之间联接的强弱，按外部的激励信号做自适应变化，而每个神经元又随着所接收到的多个信号的综合大小而呈现兴奋或抑制状态。

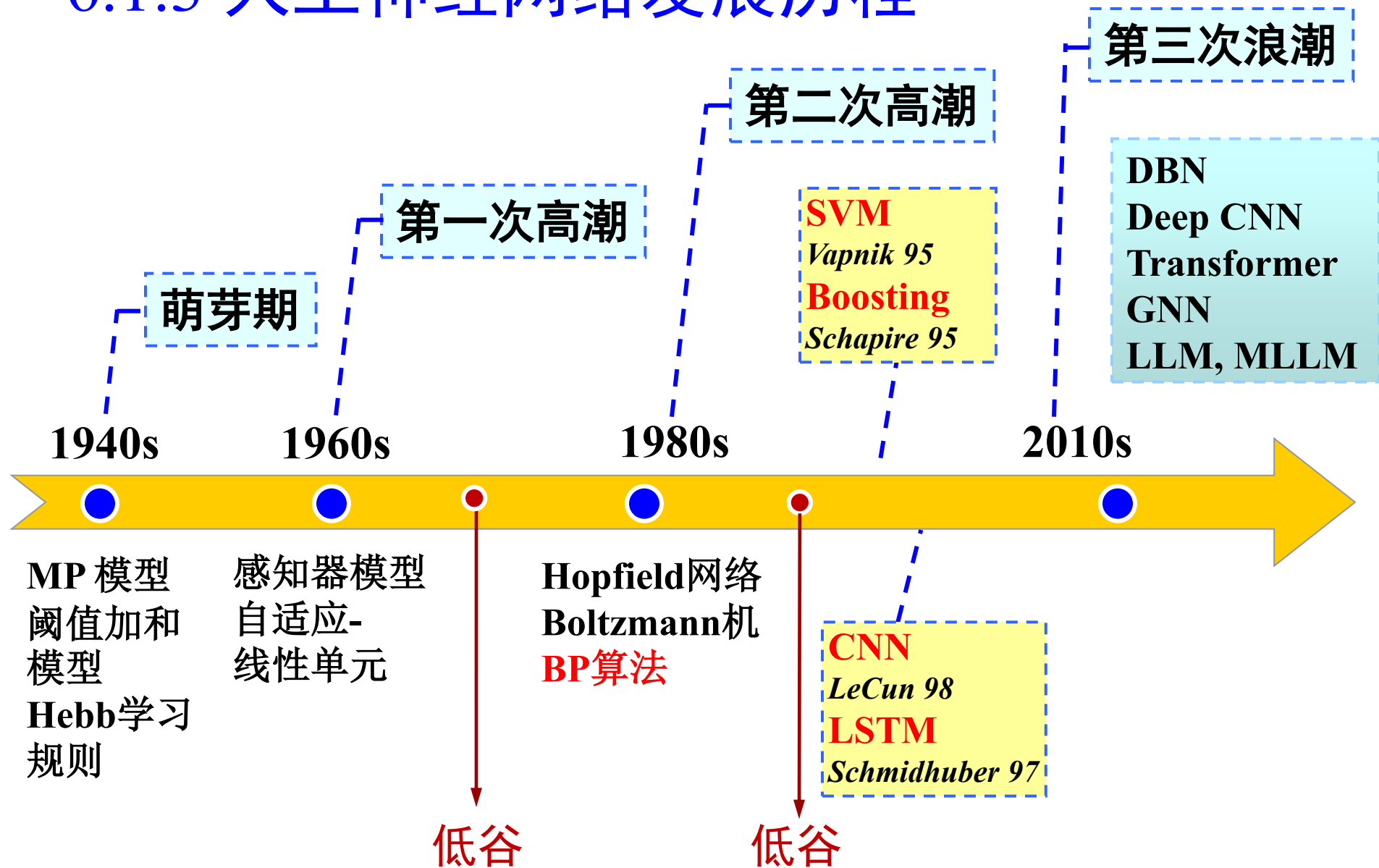
现已明确大脑的学习过程就是神经元之间连接强度随外部激励信息做自适应变化的过程，而大脑处理信息的结果则由神经元的状态表现出来。

6.1.2 什么是人工神经网络

- 按美国神经网络学者Nielsen的定义：
 - 人工神经网络是一个**并行、分布式处理结构**，由处理单元及称为联接的通道互连而成。
 - 这些处理单元具有局部内存，**可以完成局部操作**，该操作由输入至该单元的信号值和存储在该单元中的信号值来确定。
 - 每个处理单元**有一个单一的输出联接**，输出信号可以是任何需要的数学模型。

是一种模仿生物神经网络的结构和功能的数学模型或计算模型

6.1.3 人工神经网络发展历程



6.1.3 人工神经网络发展历程

- 萌芽期

- 1943年，心理学家 Warren. S. McCulloch 和数理逻辑学家 Walter Pitts 建立了神经网络的数学模型，称为 MP (Machine Perception) 模型：

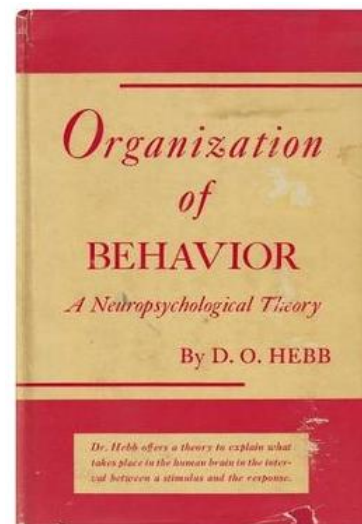
- 是一篇关于神经元如何工作的开拓性文章：“A Logical Calculus of Ideas Immanent in Nervous Activity”。
 - 指出：脑细胞的活动像断/通开关，脑细胞可以按各种方式相互结合，进行各种逻辑运算。
 - 用电路实现了简单的神经网络模型，并预言大脑的所有活动最终将被解释清楚。
- 虽然问题并非如此简单，但它给人们一个信念：即大脑的活动是靠脑细胞的组合连接实现的。

6.1.3 人工神经网络发展历程

- 萌芽期

- 1949年，心理学家 Donald Hebb 写了一本书：“**The Organization of Behavior**”。

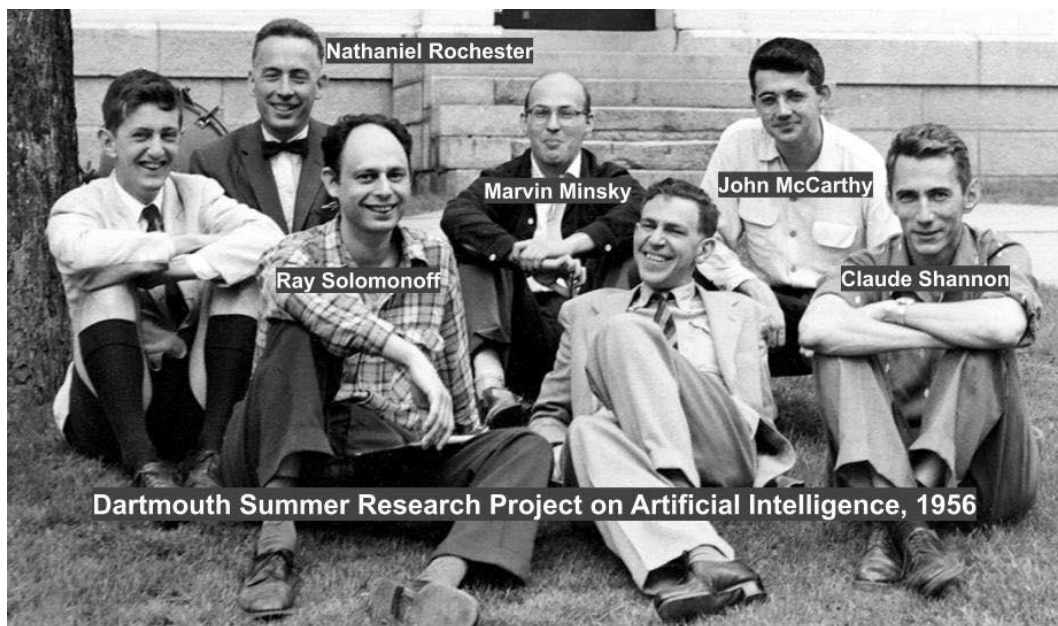
- 书中提出了突触联系强度可变的设想。
 - 强调了心理学和生理学之间的联系和沟通，指出脑细胞间的通路在参与某种活动时将被加强，这就是后来的Hebb学习规则。
 - 目前仍有一些神经网络模型仍然采用这种学习规则。



6.1.3 人工神经网络发展历程

- 萌芽期

- 1956 年，M. Minsky, H. Simon以及J. McCarthy等人在“达特茅斯夏季人工智能研究项目”首次提出了人工智能的概念，使人工智能成为计算机科学的一个分支。



Dartmouth Summer Research Project on
Artificial Intelligence, 1956

6.1.3 人工神经网络发展历程

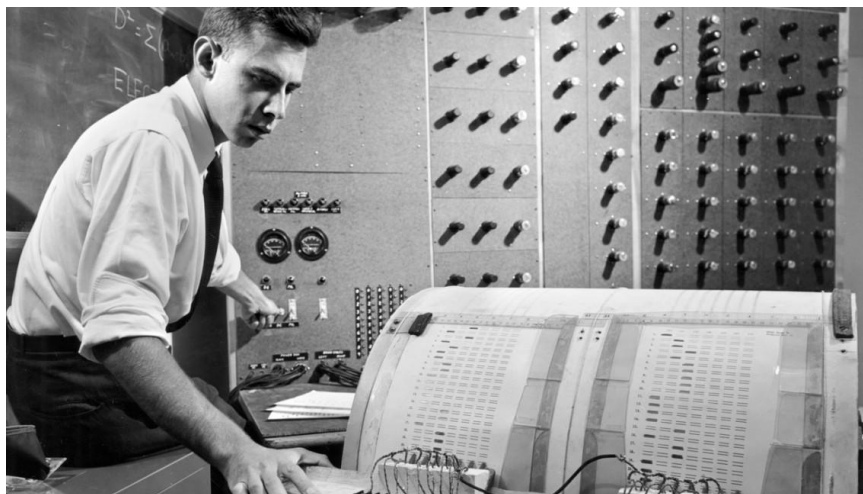
- 萌芽期

- 1956 年，M. Minsky, H. Simon以及J. McCarthy等人在“达特茅斯夏季人工智能研究项目”首次提出了人工智能的概念，使人工智能成为计算机科学的一个分支。
- 人们提出三条研究思路：
 - 符号主义：采用高级人工智能方法，试图建立描述智能机功能的计算机程序
 - 连接主义：根据大脑对初级信号（比如视觉信号）处理方式构建成结构模型，以实现智能化
 - 行为主义：类比生物系统的适应和进化，注重通过环境交互学习

6.1.3 人工神经网络发展历程

- 第一次高潮

- 1957年，计算机专家 **Frank Rosenblatt** 开始从事感知器研究，并制成硬件，**被认为是最早的神经网络模型**。



- 1959年，Bernard Widrow 和 Marcian Haff 提出一种称为自适应线性单元 (ADALINE) 的网络模型，并在论文 “Adaptive Switching Circuits” 中描述了该模型和学习算法 (**Widrow-Haff算法，见5章**)。该网络通过训练，可以用于抵消通信中的回波和噪声，也可用于天气预测，成为第一个用于实际问题的神经网络。

6.1.3 人工神经网络发展历程

- 第一次高潮
 - 1962 年，**Frank Rosenblatt** 出版了一本书 “**The Principles of Neurodynamics**”，详述了感知器模型。
 - 该感知器具有**输入层、输出层**，通过实验可以模仿人的某些特性，并断言它可以学会任何它可以表示的功能。
 - 1967年，Stephen Grossberg 通过对生理学的研究，开发了一种称作 Avalanche（**雪崩网**）的神经网络模型，可以执行连续语音识别和控制机器人手臂的运动。

6.1.3 人工神经网络发展历程

- 第一次高潮

- 在这一时期，由于感知器的某些进展和对神经网络的宣传，人们乐观地认为几乎已经找到了实现智能的关键，许多部门开始大批地投入此项研究，希望尽快占领制高点，形成了研究人工神经网络的第一次高潮。
- 由于当时对神经网络的乐观情绪的影响，人们夸大了神经网络的潜力（有人甚至担心制造机器人的人类会很快受到机器人的攻击）。

6.1.3 人工神经网络发展历程

- 反思期

- M. Minsky 等人分析了以感知器为代表的神经网络系统的功能及局限后，于1969年出版了“Perceptrons: An Introduction to Computational Geometry”一书，指出感知器不能解决高阶谓词问题，不能解决异或问题，将Rosenblatt的感知器拉下神坛。
- 他们的论点极大地影响了神经网络的研究，加之当时串行计算机和人工智能所取得的成就，掩盖了发展新型计算机和人工智能新途径的必要性和迫切性，使人工神经网络的研究处于低潮。
- 在此期间，仍有一些有影响力的工作：自适应谐振理论（ART网）、自组织映射、认知机网络，以及有关神经网络数学理论的研究。

6.1.3 人工神经网络发展历程

- 反思期

- 二十世纪70年代到80年代早期，仍有一些坚信神经网络的人坚持研究工作。比如：
 - 神经生理学家 James Anderson 开发的盒中脑模型（Brain-State-in-a-Box, BSB）。
 - 日本学者 Kuniyik Fukushima (福岛邦彦) 开发的用于视觉图形识别的认知器模型（Neocognitron, 1979）。
 - 电气工程师 Teuvo Kohonen 开发的与BSB类似的网络模型。
 - 以及 Grossberg, Rumelhart, McClelland, Marr, Amari和 Cooper 等人的工作。

6.1.3 人工神经网络发展历程

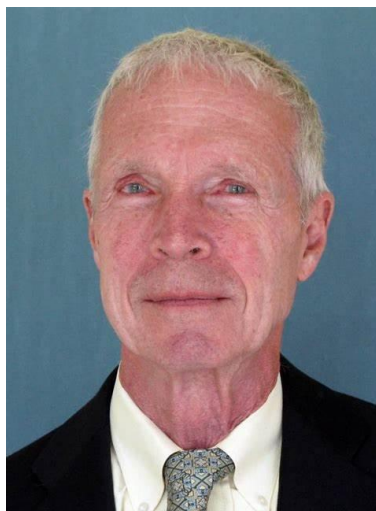
- 第二次高潮

- 1982年，John Hopfield 向美国科学院递交了有关神经网络的报告，主要内容就是建议收集和重视以前对神经网络的工作，其中特别强调了每种模型的实用性。
- 根据对神经网络的数学分析和深入理解，Hopfield 揭示了以往的网络是如何工作的，可以做些什么，并提出了他自己的模型，能从失真的或不完善的数据图像中获得完整的数据图像，引起了美国军方的兴趣。
- 当时，人工智能对自动制导车的研究失败，而利用神经网络有可能解决这个问题，使人们的注意力重新投向人工神经网络，导致了人工神经网络的第二次高潮。

6.1.3 人工神经网络发展历程

- 第二次高潮

- J. Hopfield提出Hopfield网络 (1982)
- Hinton, Sejnowsky等人提出了著名的Boltzmann机 (1985)
- Rumelhart, Hinton等人提出多层网络的学习算法—BP算法 (1986)
 - 1970s – efficient error backpropagation, Linnainmaa



John Hopfield



David Rumelhart



Geoffrey Hinton

- J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. PNAS, 1982.
- D. H. Ackley, G. E. Hinton, and T. J. Sejnowski. A learning algorithm for Boltzmann machines. Cognitive Science, 1985.
- D. E. Rumelhart, G. E. Hinton and R. J. Williams. Learning representations by back-propagating errors. Nature, 1986.

6.1.3 人工神经网络发展历程

- 第二次反思期(再认识与应用研究)
 - 二十世纪90年代后，神经网络的研究趋于平缓：
 - 应用领域不广，结果不够精确、不够稳定等问题
 - 主要研究内容
 - 开发现有模型的应用，改进模型和算法，提高训练速度和准确度。
 - 期望在理论上寻找新的突破，建立新的专用或通用模型和算法。
 - 进一步对生物神经网络进行研究，不断丰富对人脑的认识

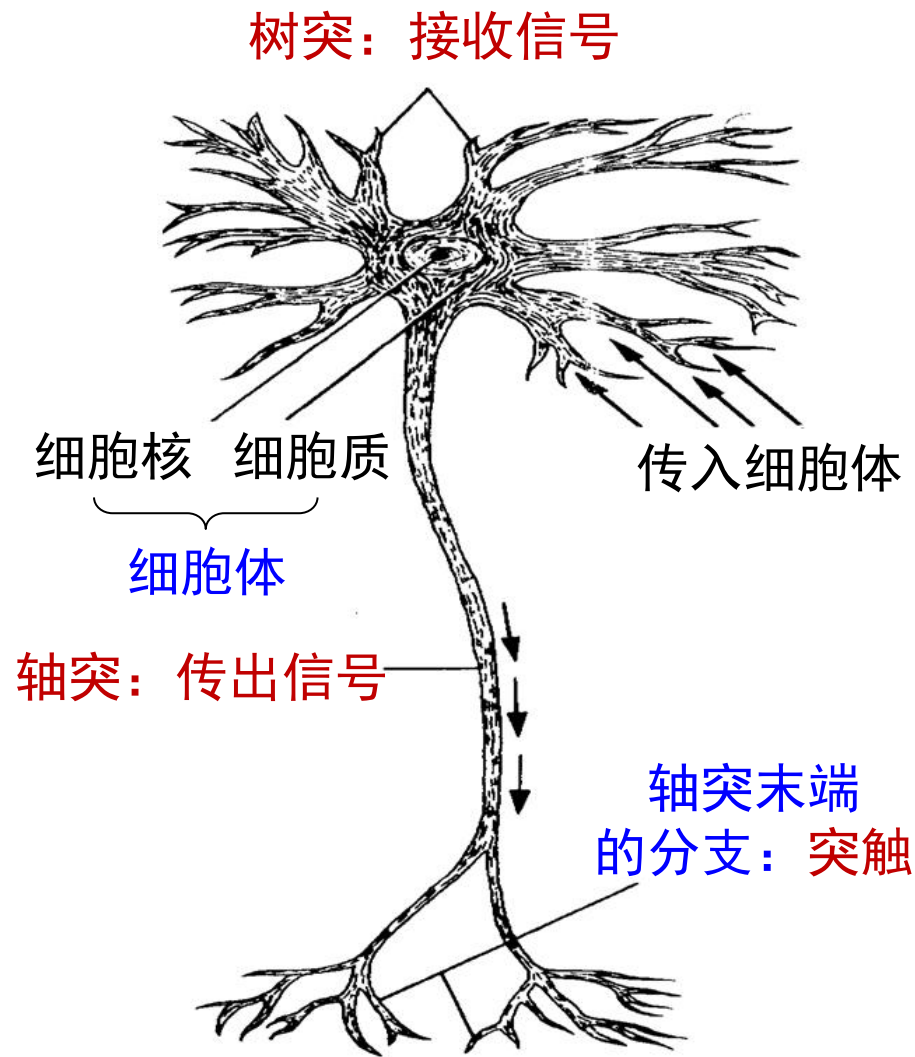
第二节

人工神经网络基础

6.2.1 神经元

- 生物学原型

神经细胞由**细胞体**和**突起**组成。突起分为两种：**短的叫树突**，从其它神经细胞接收信号传递给细胞体；**长的叫轴突**，将信号传递给其它神经细胞的树突。



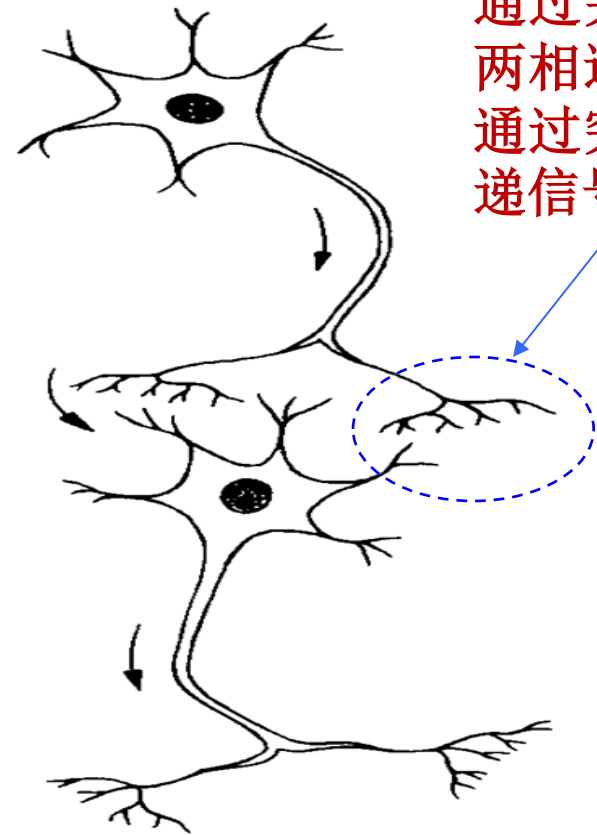
6.2.1 神经元

细胞体：它是神经元的本体，内有细胞核和细胞质，完成普通细胞的生存功能。

树突：它有大量的分枝，多达 10^3 数量级，长度较短（通常不超过1毫米），用以接收来自其它神经元的信号。

轴突：它用以输出信号，有些较长（可达1米以上），轴突的远端也有分枝，可与多个神经元相连。

突触：它是一个神经元与另一个神经元相联系的特殊部位，通常是一个神经元轴突的端部靠化学接触或电接触将信号传递给下一个神经元的树突或细胞体。

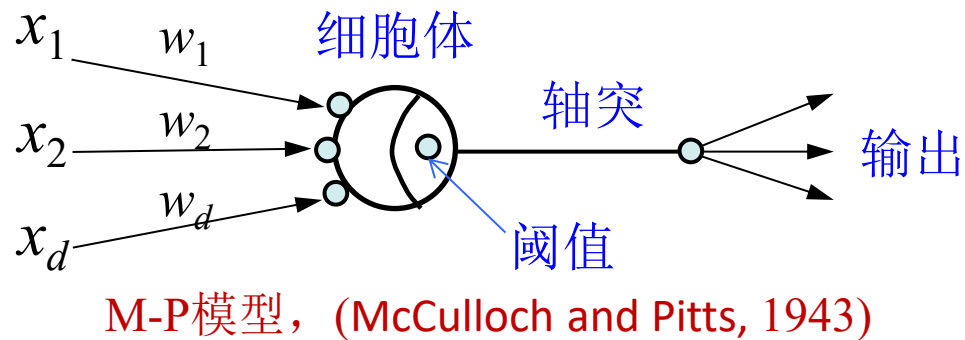


神经元之间通过突触两两相连，并通过突触传递信号。

6.2.1 神经元

- 神经元的基本工作机制
 - 一个神经元有两种状态：兴奋和抑制
 - 平时处于抑制状态的神经元，当接收到其它神经元经由突触传来的冲击信号时，多个输入在神经元中以代数和的方式叠加
 - 进入突触的信号会被加权，起兴奋作用的信号为正，起抑制作用的信号为负
 - 如果叠加总量超过某个阈值，神经元就会被激发进入兴奋状态，发出输出脉冲，并由轴突的突触传递给其它神经元

6.2.2 人工神经元



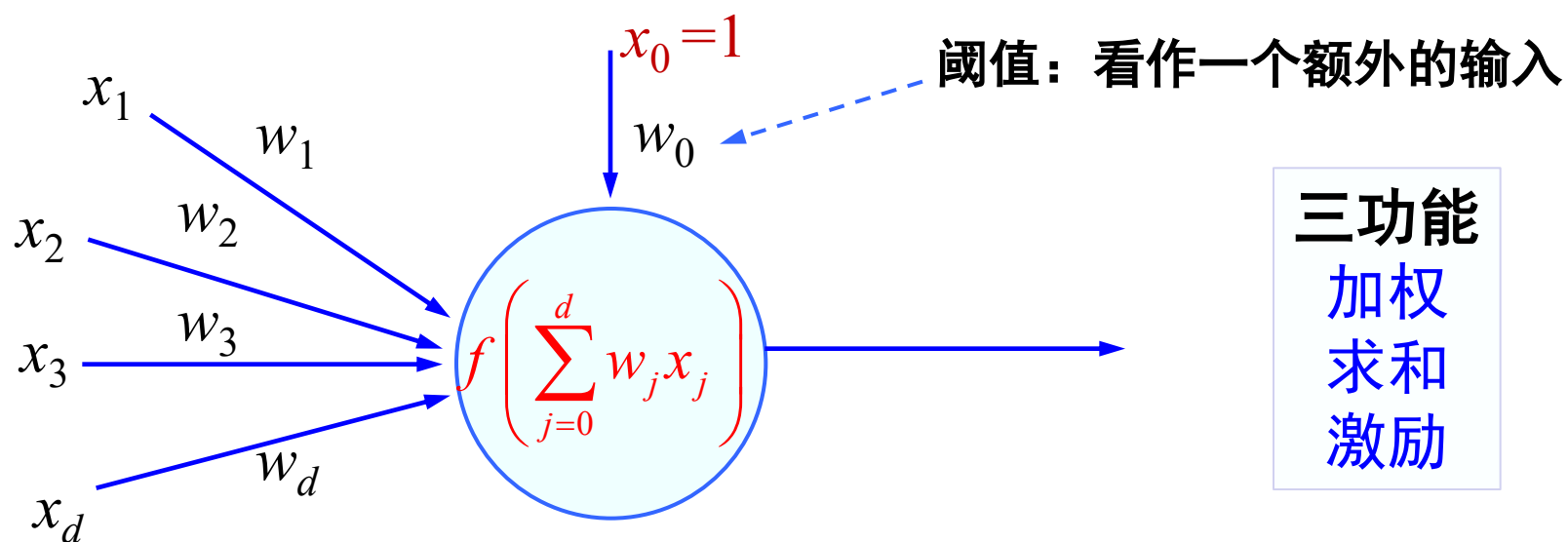
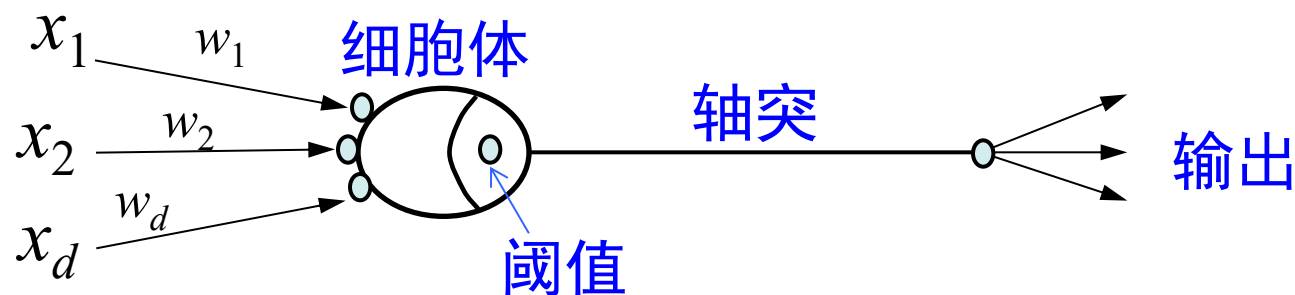
- 对生物神经元的模拟

- 与生物神经元一样，处理单元也有很多输入信号，并同时叠加到处理单元上。
- 处理单元以输出作为响应。输出响应不但受输入信号的影响，同时也受内部因素的影响：
 - 内部因素：内部阈值或一个额外输入（称为偏置项）
- 处理单元的每一个输入都经过相关的加权，以影响输入的激励作用：
 - 类似于生物神经元中突触的可变强度，权重确定了输入信号的强度。
 - 类似于生物神经元中的突触强度可受外界因素影响，权重可以调节。

6.2.2 人工神经元

- 处理单元的功能
 - 对每个输入信号进行加权处理（确定其强度）；
 - 确定所有输入信号的叠加效果（求和）；
 - 确定其输出（转移特性，即激励特性）。

- 处理单元的基本结构和功能



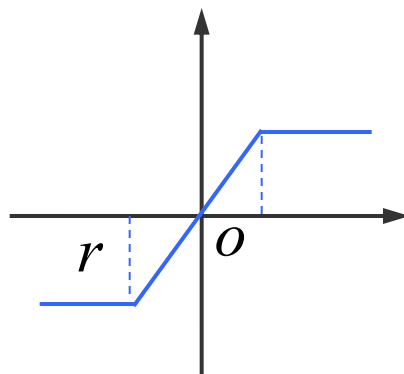
$\mathbf{x}$: 收到信号 $\mathbf{w}$: 信号的权重 $f(\cdot)$: 激活函数

6.2.2 人工神经元

- 激活函数

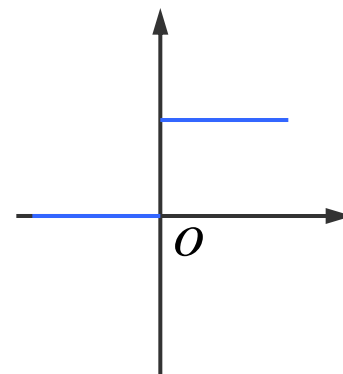
- 激活函数 $f(\cdot)$ ，也称激励函数、转移函数、传输函数或限幅函数，其作用是**将可能的无限域变换到指定的有限范围内进行输出，并引入非线性。**
- 常用的激活函数：
 - 线性函数、斜坡函数、阶跃函数、符号函数、Sigmoid函数、双曲正切函数、ReLU

$$f(s) = \begin{cases} r, & s > r \\ s, & |s| \leq r \\ -r, & s < -r \end{cases}$$

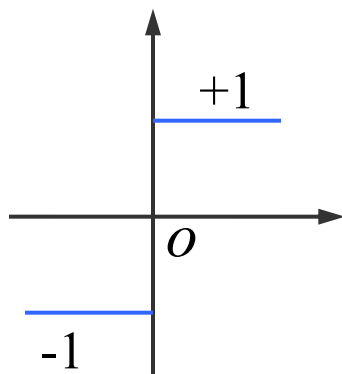


斜坡函数

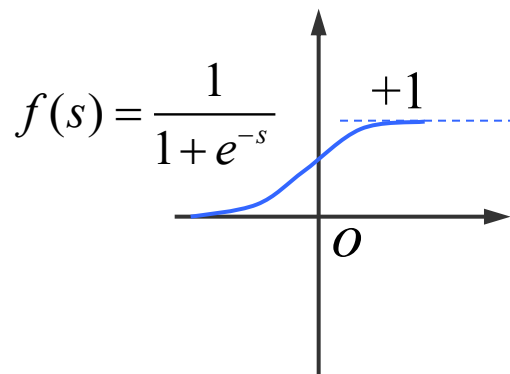
$$f(s) = \begin{cases} 1, & s \geq 0 \\ 0, & s < 0 \end{cases}$$



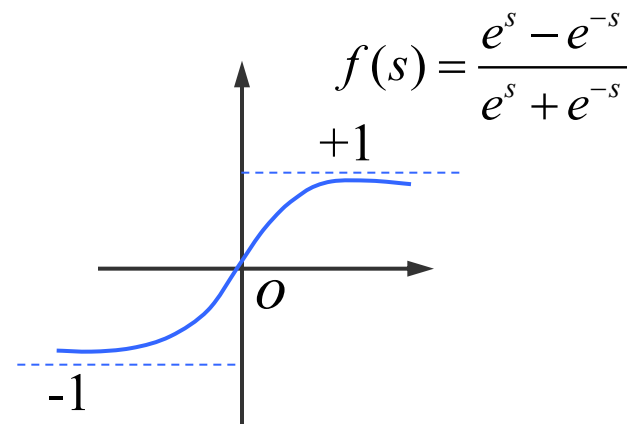
阶跃函数



符号函数



Sigmoid函数



双曲正切函数(tanh)

激活函数

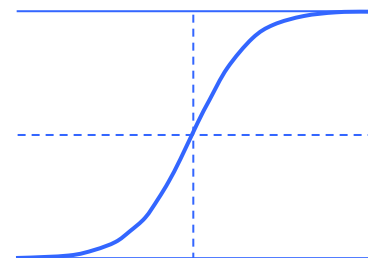
- 软饱和函数性质：

$$\lim_{s \rightarrow \infty} f'(s) = 0$$

- 左饱和：
$$\lim_{s \rightarrow -\infty} f'(s) = 0$$

- 右饱和：
$$\lim_{s \rightarrow +\infty} f'(s) = 0$$

$$f(s) = \frac{1}{1 + e^{-s}}$$

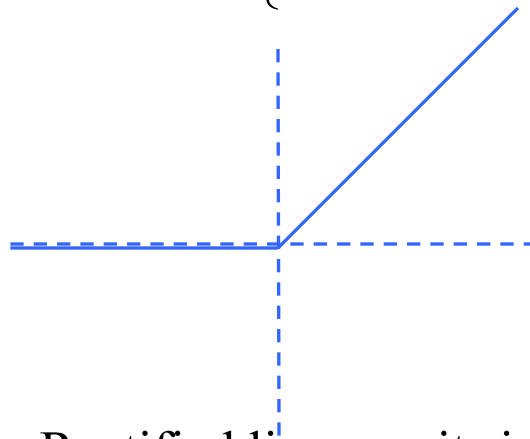


- Sigmoid 函数的软饱和性，使得深度神经网络在二三十年里一直难以有效的训练，**是阻碍神经网络发展的重要原因。**
- 硬饱和函数：与软饱和相对的是硬饱和激活函数，即： $f'(x)=0$ ，当 $|x| > c$ ，其中 c 为常数。同理，硬饱和也分为左饱和和右饱和。

激活函数

- 目前，深度学习在诸多领域取得了引人注目的成果。其中，激活函数的发展也起到了重要的作用。
- 新型激活函数ReLU在一定程度上克服了梯度消失，使得深度网络的直接监督式训练成为可能。

$$f(s) = \begin{cases} s & s \geq 0 \\ 0 & s < 0 \end{cases}$$



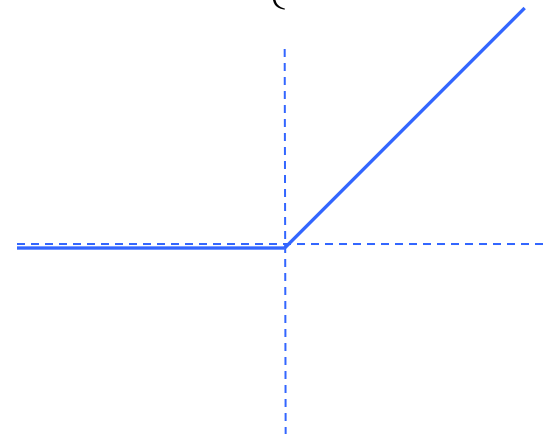
V. Nair and G. E. Hinton. Rectified linear units improve restricted Boltzmann machines. In ICML, 2010.

激活函数

- ReLU

- ReLU 在 $x < 0$ 时硬饱和。由于 $x > 0$ 时导数为 1，所以，ReLU 能够在 $x > 0$ 时保持梯度不衰减，从而缓解梯度消失问题。
- 导致网络稀疏性
- 但随着训练的推进，部分输入会落入硬饱和区，导致对应权重无法更新。这种现象被称为“**神经元死亡**”。
- ReLU还经常被“诟病”的一个问题是输出具有**偏移现象**，即输出均值恒大于零。
- 偏移现象和神经元死亡会共同影响网络的收敛性。

$$f(s) = \begin{cases} s & s \geq 0 \\ 0 & s < 0 \end{cases}$$

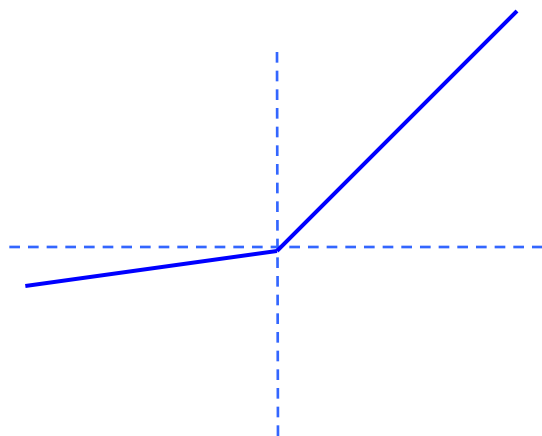


激活函数

- ReLU的进一步改进

Leaky ReLU

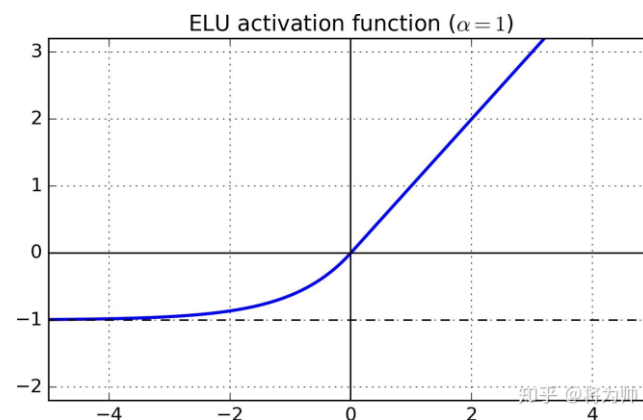
$$f(s) = \begin{cases} s & s \geq 0 \\ \alpha s & s < 0 \end{cases}$$



解决神经元死亡

ELU

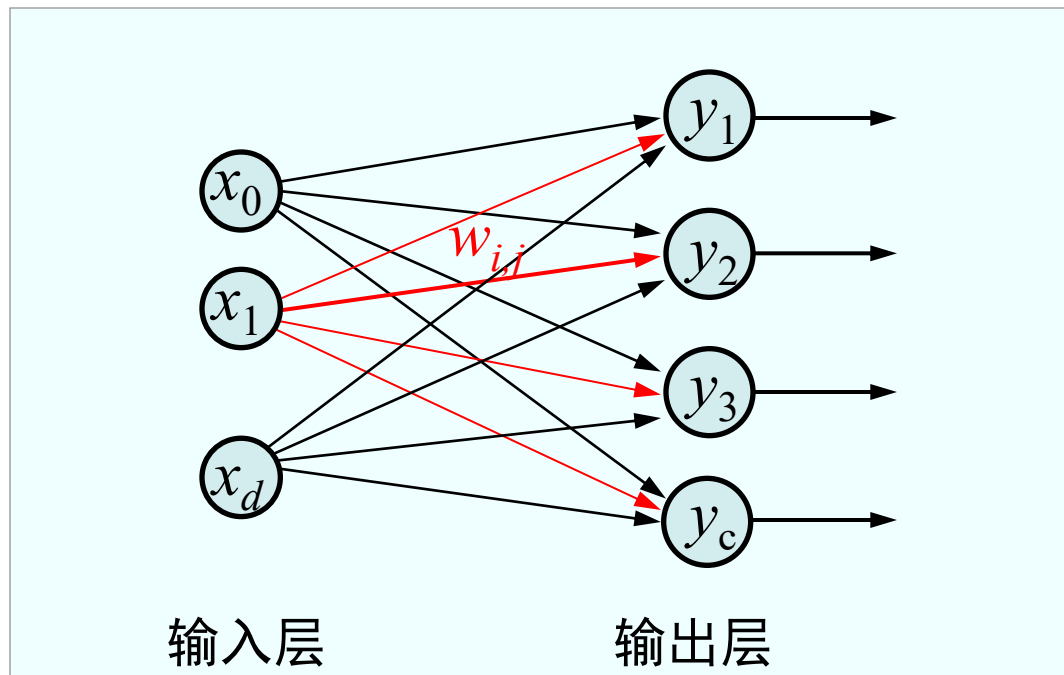
$$f(s) = \begin{cases} s & s \geq 0 \\ \alpha(e^s - 1) & s < 0 \end{cases}$$



解决输出偏移

6.2.3 拓扑结构

- 前馈神经网络：单层网络



净输入： $net_j = \sum_{i=0}^d w_{ij} x_i$

输出： $y_j = f(net_j)$

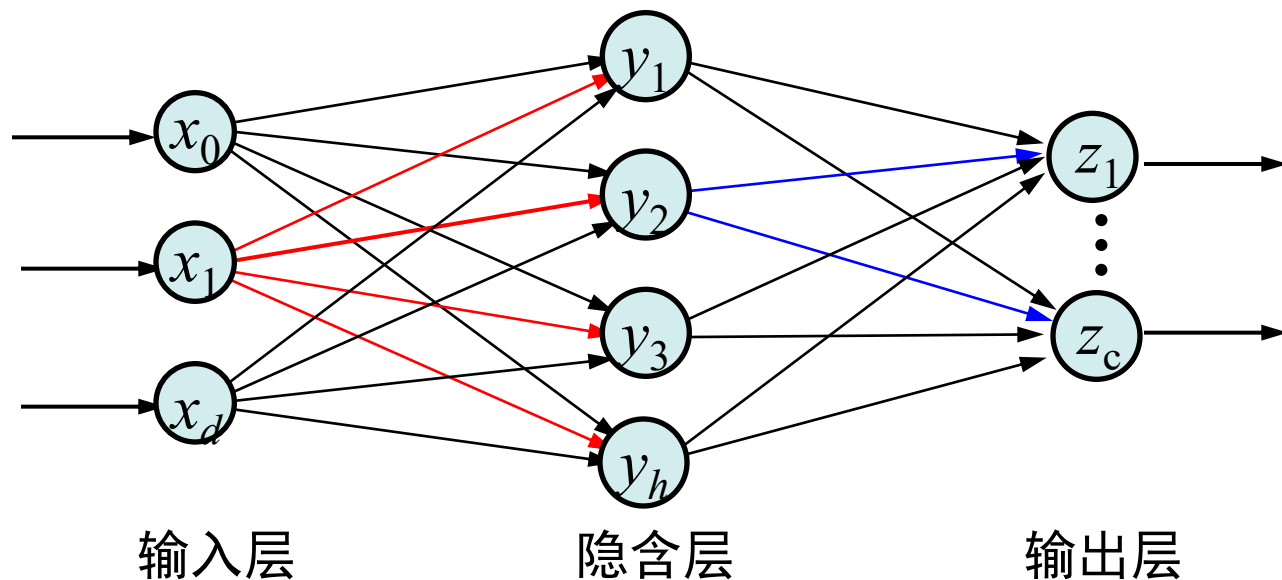
矩阵形式： $\mathbf{y} = f(\mathbf{W}\mathbf{x}) \in R^c$

$(d+1)*c$ 个
权重参数

6.2.3 拓扑结构

- 前馈神经网络：多层网络

- 虽然目前有很多网络模型，但它们的结点基本上都是按层排列的。
- 多层网络是由单层网络进行级联构成的，即上一层的输出作为下一层的输入。



$$\mathbf{z} = f(\mathbf{W}_2 f(\mathbf{W}_1 \mathbf{x})) \in R^c$$

$(d+1)*h+h*c$ 个
权重参数

6.2.3 拓扑结构

- 前馈神经网络：多层网络
 - 接收输入信号的层称为输入层。
 - 产生输出信号的层称为输出层。
 - 中间层，又称为隐含层，不直接与外部环境打交道。
 - 隐含层的层数可从零到若干层。
 - 层与层之间可以全连接、也可以部分连接的情况。

6.2.3 拓扑结构

- 前馈神经网络：多层网络
 - 激活函数应是非线性的，否则多层网络的表示能力并不比单层网络强

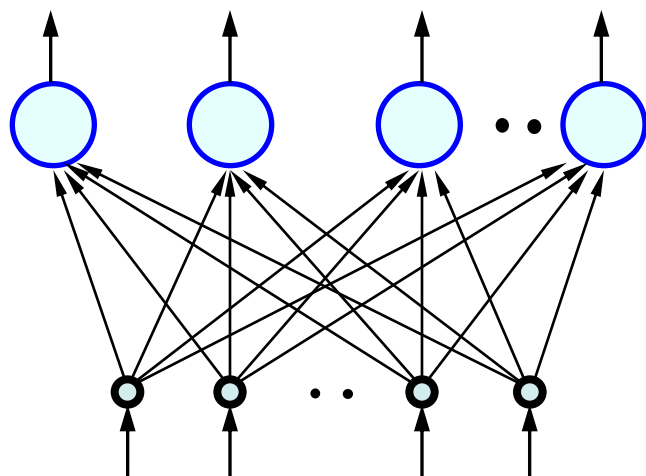
- 对线性转移函数：第一层的输出 $\mathbf{W}_1 \mathbf{x}$ 作为第二层的输入，通过第二个加权矩阵得到网络的输出：

$$\mathbf{y} = \mathbf{W}_2 (\mathbf{W}_1 \mathbf{x}) = (\mathbf{W}_2 \mathbf{W}_1) \mathbf{x} = \mathbf{W} \mathbf{x}$$

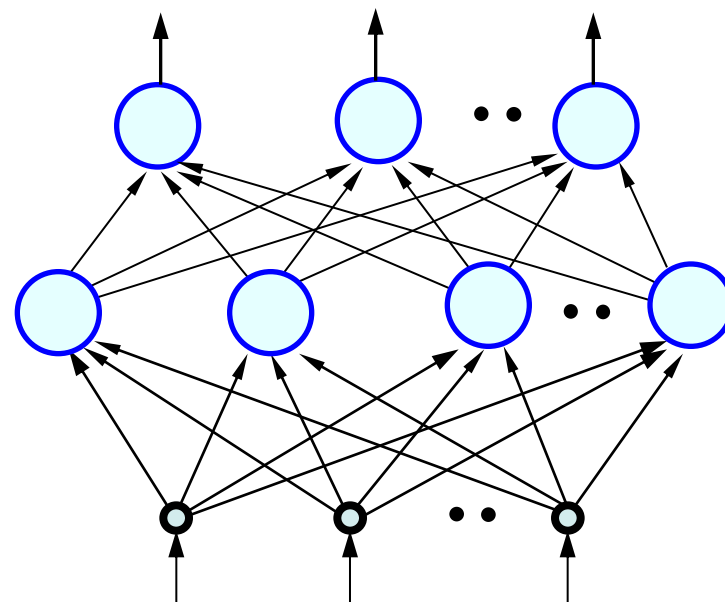
- 可见，两层线性网络等效于单层网络，只是后者的加权矩阵为两个加权矩阵的乘积。

6.2.3 拓扑结构

- 前馈神经网络（全连接）



单层感知器
(Perceptron)

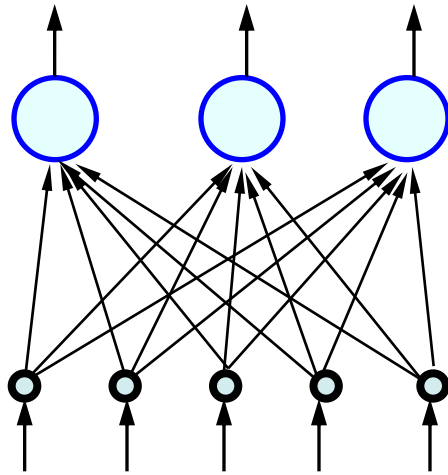


多层感知器
(Multi-Layer Perceptron, MLP)

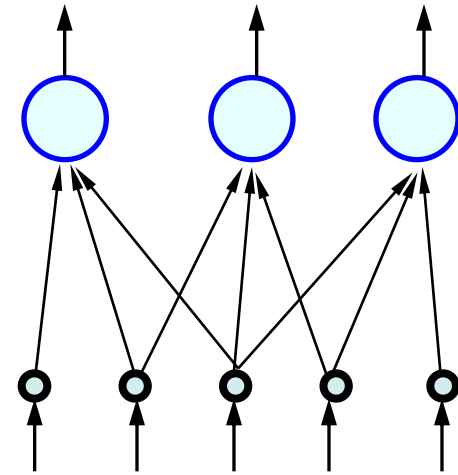
没有层内联接，各结点前馈联接到下一层所有结点

6.2.3 拓扑结构

- 全连接 vs 部分连接



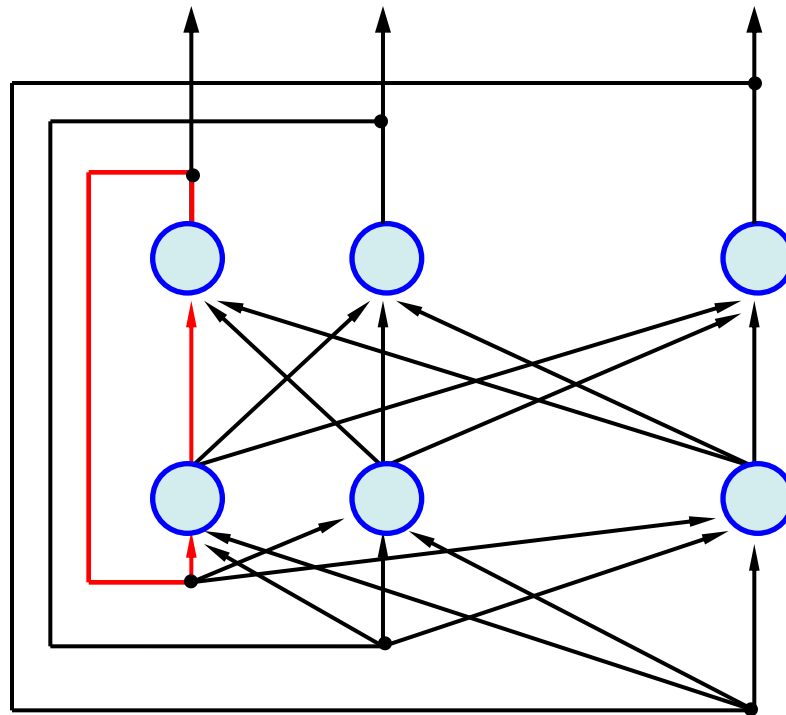
全连接



部分（局部）连接

6.2.3 拓扑结构

- 反馈网络



网络有回路，信息可以沿回路流动

结点的输出依赖于当前的输入，也依赖于自己以前的输出。

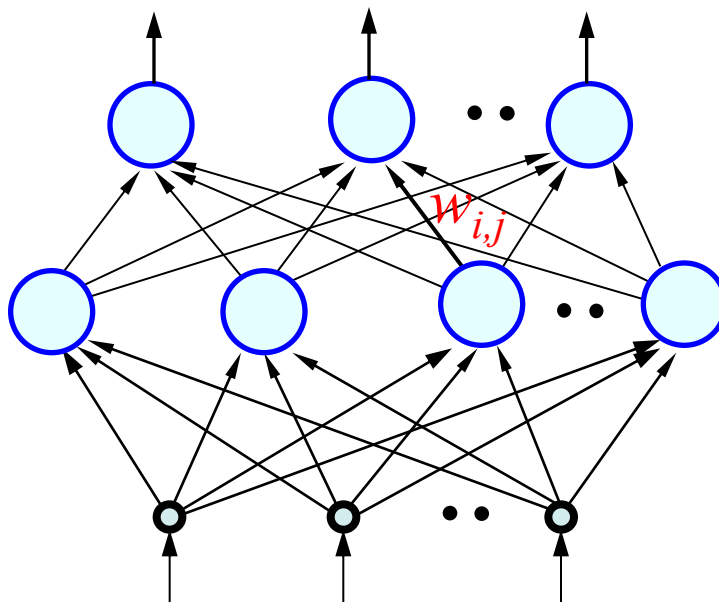
6.2.3 拓扑结构

- 前馈网络与反馈网络的比较
 - 前馈型网络“不存储短期记忆”，结点的输出仅仅是当前输入的加权和。
 - 在反馈网络中，要将以前的输出循环返回到输入。
 - 反馈网络类似于“人类的短期记忆”，即网络的输出状态部分取决于以前的输入，是一类广泛应用的网络。

6.2.4 网络训练

- 任务

- 给定 n 个观测数据 $\{\mathbf{x}_k\}$ ，训练网络的各层结点之间的连接权重 w_{ij} (含偏置项)



6.2.4 网络训练

- 有监督的训练

- 希望训练这样一个网络，对每个训练数据，通过该网络计算之后尽可能输出其给定的目标值。

- 无监督训练

- 无监督的训练不要求有目标向量，其目的是训练一个网络，使其产生的输出具有某种可理解的规律性。
 - 从本质上讲，训练过程是抽取样本所隐含的统计特征。

6.2.4 网络训练

Hebb认为：如果源和目的神经元都被激活或抑制，则它们之间的联系会加强。

- Hebb训练方法（经验方法）

- **基本思路（突触修正假设）**：如果两个相互联接的处理单元（结点）处于相同的激励电平（即输出值具有相同的符号），则突触传导增强（权重增强）。
- **训练策略**：数学上，两结点 i 和 j 的连接权重将按它们的输出值之积来改变：

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} + \eta y_i z_j$$

权重更新 训练速率 结点 i, j 的输出

- Hebb学习规则需预先设置权饱和值，以防止输入和输出正负始终一致时出现权重无约束增长。
- Hebb学习规则是前馈、无监督学习规则。

6.2.4 网络训练

- δ 训练方法（分析方法）
 - **基本思路**：按差值（ δ 值）最小准则修正各连接权重的强度。
 - 差值最小：差值是指处理单元所要求的输出与当前实际输出间的差值，通过调节各权重值以达到最小。
 - **训练策略**：梯度下降法
 - 普遍采用的一种有效的监督训练方法：

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

与训练样本相关的损失函数
或能量函数

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} + \Delta w_{ij}$$

6.2.4 网络训练

- 随机训练方法

- **基本思路**：采用概率和能量关系来调节权重
- **训练策略**：**随机改变一个权重**，计算改变后产生的最终能量，并按如下准则来确定是否接受此改变：
 - 若改变后，网络的能量降低，则接受这一改变。
 - 若改变后，能量没有降低，则根据一个预定的概率分布来保留（接受）这一改变。
 - 否则，拒绝这一改变，使权值恢复到原来的值。
- 典型的随机训练算法：**模拟退火算法**

6.2.4 网络训练

- Kohonen训练方法（竞争学习）

- 基本思路：

- 在训练过程中结点（处理单元）参与彼此竞争，具有最大输出的结点为获胜者。
 - 获胜的结点具有抑制其竞争者的能力和激活其近邻结点的能力，只有获胜者和其近邻结点的权重才被允许调节。
 - 获胜者的近邻结点的范围在训练中是可变的。

- 自组织竞争型神经网络(Self-Organizing Maps, SOM)

芬兰 Teuvo Kohonen 教授受生物系统的启发而提出，是无监督训练方法。



Frank Rosenblatt

第三节

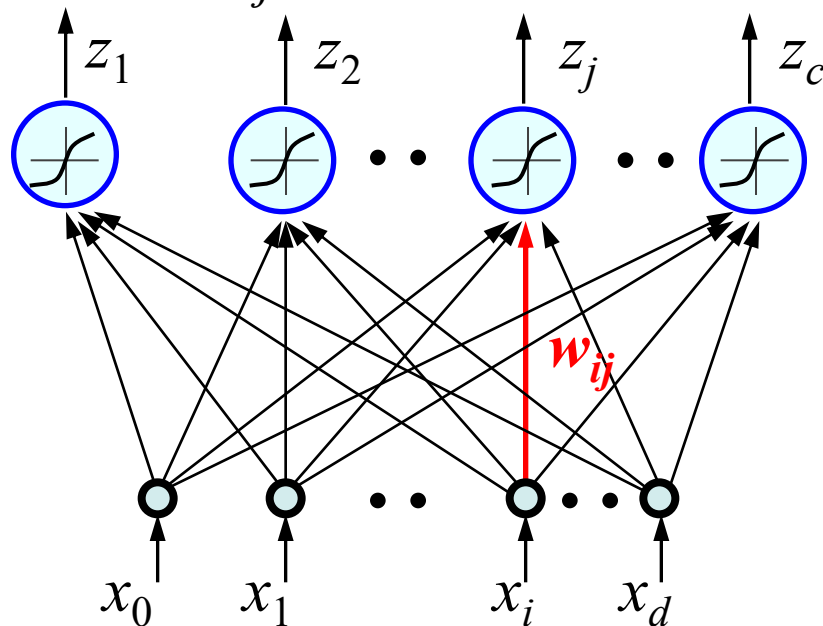
单层前馈神经网络

(单层感知器, Perceptron)

6.3.1 感知器的结构

- 单层感知器网络描述

- 由含有 $d+1$ 个结点（包含 bias 结点）输入层和含有 c 个结点的输出层构成。
- 输入向量： $\mathbf{x}=[1, x_1, x_2, \dots, x_d]^T$ （齐次增广）
- 输出向量： $\mathbf{z}=[z_1, z_2, \dots, z_c]^T$
- 权重集合： $\mathbf{W}=\{w_{ij}\}$



6.3.1 感知器的结构

- 单层感知器网络描述
 - 对于输出结点 j :

$$net_j = \sum_{i=0}^d w_{ij} x_i = \mathbf{w}_j^T \mathbf{x},$$

$$z_j = f(net_j) = f\left(\sum_{i=0}^d w_{ij} x_i\right)$$

- 其中, $\mathbf{w}_j = [w_{0j}, w_{1j}, w_{2j}, \dots, w_{dj}]^T$
- $f(\cdot)$ 为激活函数, 不同激活函数的感知器具有不同的功能。

6.3.2 感知器的训练

- 学习任务

- 任务：确定网络各个联接权重 $\{w_{ij}\}$

- 给定训练样本 $\{\mathbf{x}^k\}$ 及其目标值 $\{\mathbf{t}^k\}$ ，当网络训练完成后（即权重均确定），希望：

$$\mathbf{z}^k \approx \mathbf{t}^k, k = 1, 2, \dots, n \quad (\mathbf{z}^k \text{ 为网络对 } \mathbf{x}^k \text{ 的输出})$$

- 对分类问题：

- 两类：输出层只需一个结点。目标值为+1或-1。
 - c 类：输出层需要 c 个结点。 $\mathbf{t}$ 是 $\mathbf{x}$ 的类别的onehot编码：
 $\mathbf{t}_j$ 为一个类别标签向量中的第 j 个元素： $\mathbf{t}_j = 1$ ，如果 $\mathbf{x}$ 属于第 j 类，否则为0。

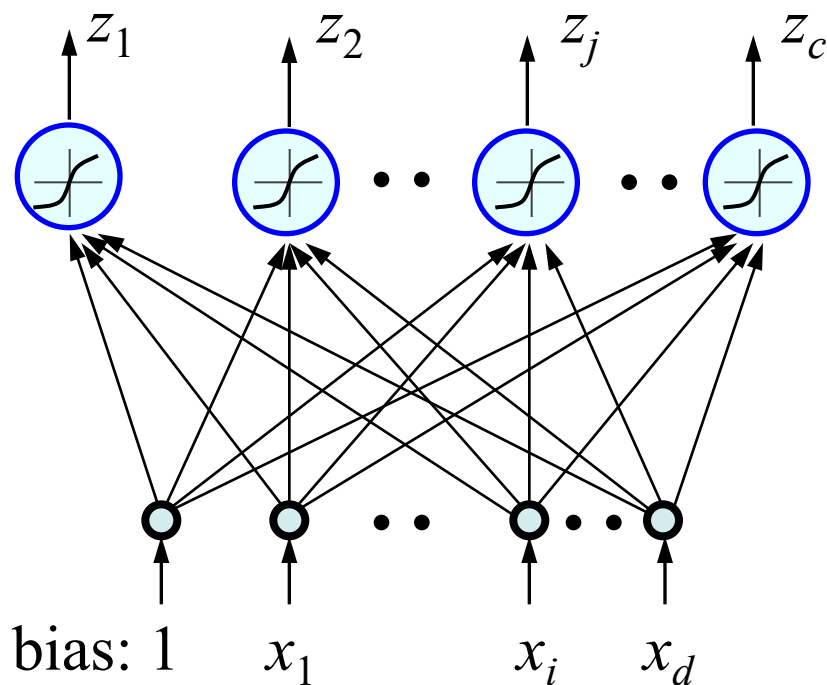
6.3.2 感知器的训练

- 学习任务

i : 联系输入层各结点
 j : 联系输出层各结点

目标:

$$z_1 \approx t_1 \quad z_2 \approx t_2 \quad z_j \approx t_j \quad z_c \approx t_c$$



6.3.2 感知器的训练

• δ 训练方法(梯度下降法)

误差函数:

$$E(\mathbf{w}) = \frac{1}{2} \sum_k \|\mathbf{t}^k - \mathbf{z}^k\|^2 = \frac{1}{2} \sum_{k,j} (t_j^k - z_j^k)^2 = \frac{1}{2} \sum_{k,j} [t_j^k - f(\text{net}_j^k)]^2 = \frac{1}{2} \sum_{k,j} [t_j^k - f(\mathbf{w}_j^T \mathbf{x}^k)]^2$$

梯度:

$$\frac{\partial E}{\partial w_{ij}} = - \sum_k \frac{\partial E}{\partial z_j^k} \frac{\partial z_j^k}{\partial \text{net}_j^k} \frac{\partial \text{net}_j^k}{\partial w_{ij}} = - \sum_k [t_j^k - z_j^k] f'(\text{net}_j^k) x_i^k$$

单个样本对参数更新的贡献:

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}} = \eta [t_j^k - z_j^k] f'(\text{net}_j^k) x_i^k$$

定义节点误差 δ_j^k 是误差函数 E 对节点 j 净输入 net_j^k 的负导数:

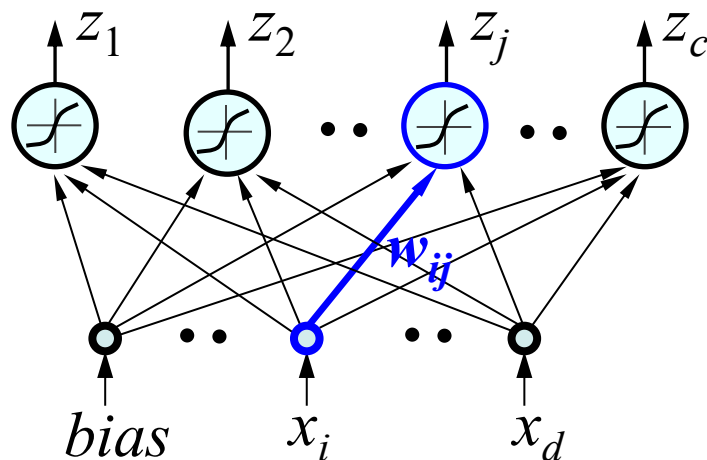
$$\delta_j^k \triangleq \frac{-\partial E}{\partial \text{net}_j^k} = f'(\text{net}_j^k) [t_j^k - z_j^k] \Rightarrow \Delta w_{ij} = \eta \delta_j^k x_i^k \quad (\delta \text{规则})$$

6.3.2 感知器的训练

- δ 训练方法(梯度下降法)

单个样本
的贡献:

$$\Delta w_{ij} = \eta \delta_j^k x_i^k, \quad \delta_j^k = \frac{-\partial E}{\partial net_j^k} = f'(net_j^k)[t_j^k - z_j^k]$$

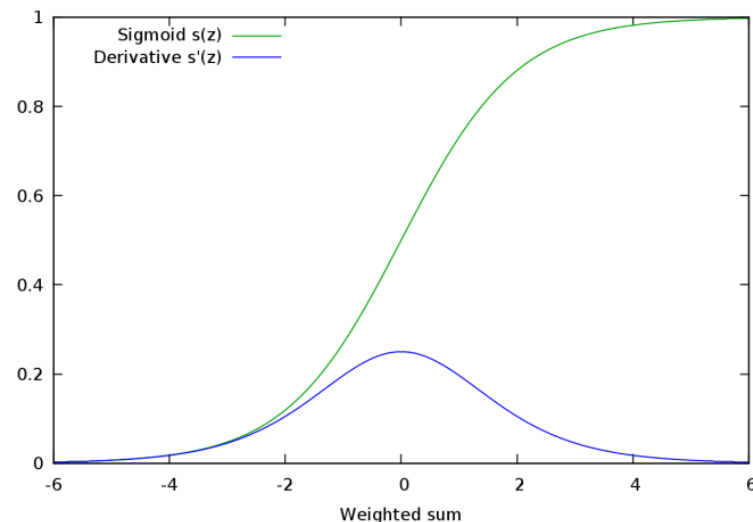


边的起始结点的输出
乘以边的指向结点的误差



6.3.2 感知器的训练

- 非线性单元
 - 例子：Sigmoid 函数



$$f(net) = \frac{1}{1 + e^{-net}}$$

$$f'(net) = \frac{e^{-net}}{(1 + e^{-net})^2} = \frac{1}{1 + e^{-net}} \left(1 - \frac{1}{1 + e^{-net}} \right) = z(1 - z), \quad z = f(net)$$

$$\delta_j^k = z_j^k (1 - z_j^k) [t_j^k - z_j^k]$$

6.3.2 感知器的训练

- 随机单样本更新算法

Stochastic Updating

- 1 begin initialize: $\mathbf{w}$, η , criterion θ , $k=0$
 - 2 do $k \leftarrow k + 1 \pmod{n}$
 - 3 randomly chose a sample $\mathbf{x}^k$
 - 4 $w_{ij} \leftarrow w_{ij} + \eta \delta_j^k x_i^k$
 - 5 until $\|\nabla J(\mathbf{w})\| < \theta$
 - 6 return $\mathbf{w}$
 - 7 end
-

适用：超大规模数据

- 批量更新算法

Batch Updating

1 begin initialize: $\mathbf{w}$, η , criterion θ , $r=0$

2 do $r \leftarrow r + 1$ (increment epoch)

3 $k = 0$, $\Delta w_{ij} = 0$

4 do $k \leftarrow k+1$

5 select the sample $\mathbf{x}^k$

6 $\Delta w_{ij} \leftarrow \Delta w_{ij} + \eta \delta_j^k x_i^k$

7 until $k = n$

8 $w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$

// 所有样本完成之后再更新

9 until $\|\nabla J(\mathbf{w})\| < \theta$

10 return $\mathbf{w}$

11 end

第四节

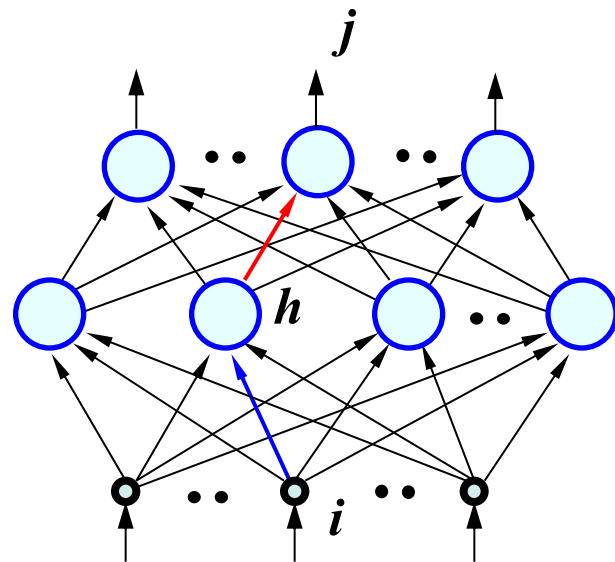
多层前馈神经网络

(多层感知器, Multi-Layer Perceptron)

6.4.1 多层感知器

- 三层网络的描述

- 训练数据输入输出对: $\{\mathbf{x}^k, \mathbf{t}^k\}$
- 输出层结点的输出: z_j^k
- 隐含层结点的输出: y_h^k
- 输入信号: x_i^k
- 输入层结点 i 至隐含层结点 h 的权重: w_{ih}
- 隐含层结点 h 至输出层结点 j 的权重: w_{hj}
- 上标 k 表示训练数据的序号, $k=1, 2, \dots, n$



- 三层网络的描述

对第 k 个样本

隐含层 h 结点的净输入:

$$net_h^k = \sum_i w_{ih} x_i^k$$

隐含层 h 结点的输出:

$$y_h^k = f(net_h^k) = f\left(\sum_i w_{ih} x_i^k\right)$$

输出层 j 结点的
净输入:

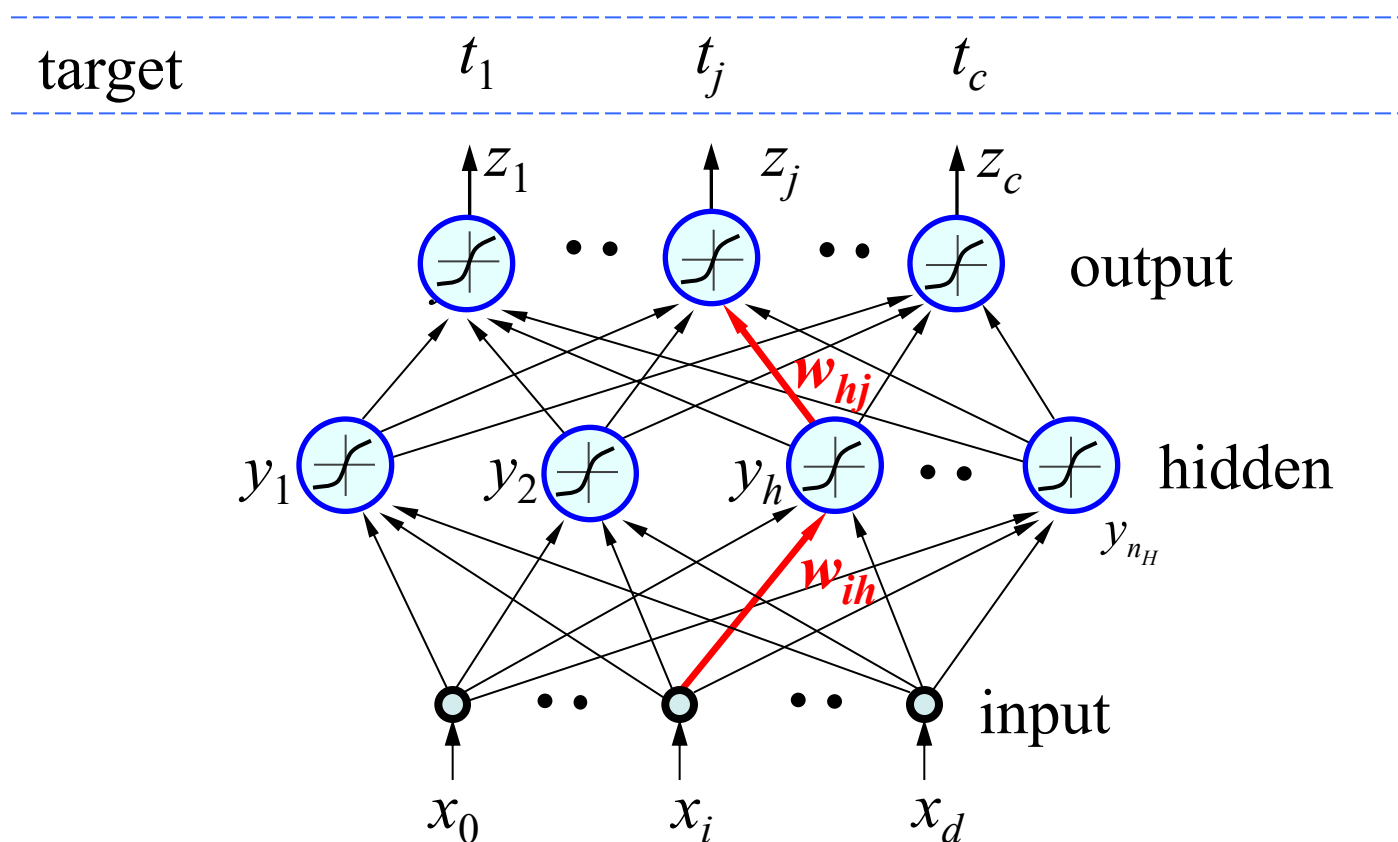
$$net_j^k = \sum_h w_{hj} y_h^k = \sum_h w_{hj} f\left(\sum_i w_{ih} x_i^k\right)$$

输出层 j 结
点的输出:

$$z_j^k = f(net_j^k) = f\left(\sum_h w_{hj} y_h^k\right) = f\left(\sum_h w_{hj} f\left(\sum_i w_{ih} x_i^k\right)\right)$$

计算: 层内并行, 层间串行

- 学习任务：确定联接权重 $\{w_{ih}\}$, $\{w_{hj}\}$
- 目标： $z_1^k \approx t_1^k, \dots, z_c^k \approx t_c^k$ for all k or $\frac{1}{2} \sum_{k,j} (t_j^k - z_j^k)^2 \approx 0$

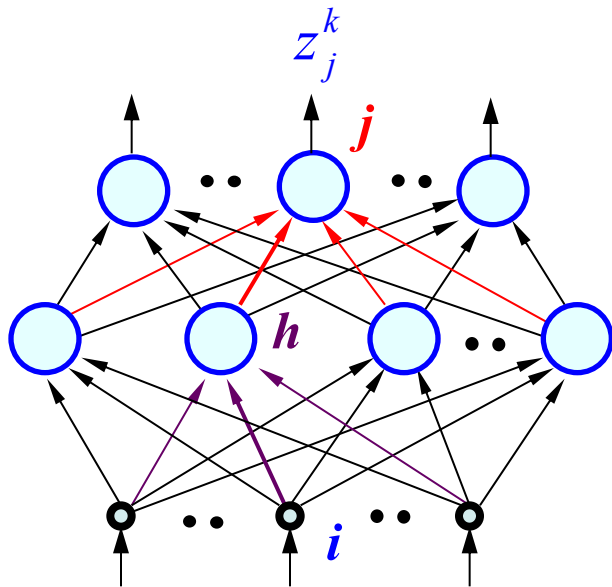


6.4.2 误差反向传播(BP)算法

- D. Rumelhart 等人于 1986 年提出了 误差反向传播 (Back Propagation, BP)学习算法
 - D. E. Rumelhart, G. E. Hinton and R. J. Williams. *Learning representations by back-propagating errors*. Nature, 1986.
- BP算法核心是梯度下降
- 基本原理
 - 利用输出层的误差来估计输出层的前一层的误差，再用这个误差估计更前一层的误差，如此一层一层地反传下去，从而获得所有其它各层的误差估计

BP是一种高效计算神经网络权重梯度的算法

- 误差函数—单个样本 x^k



$$\begin{aligned} z_j^k &= f(\text{net}_j^k) \\ &= f\left(\sum_h w_{hj} y_h^k\right) \end{aligned}$$

$$\begin{aligned} y_h^k &= f(\text{net}_h^k) \\ &= f\left(\sum_i w_{ih} x_i^k\right) \end{aligned}$$

$$\begin{aligned} E(\mathbf{w})^k &= \frac{1}{2} \sum_j (t_j^k - z_j^k)^2 \\ &= \frac{1}{2} \sum_j (t_j^k - f(\text{net}_j^k))^2 \\ &= \frac{1}{2} \sum_j \left\{ t_j^k - f\left(\sum_h w_{hj} y_h^k\right) \right\}^2 \\ &= \frac{1}{2} \sum_j \left\{ t_j^k - f\left(\sum_h w_{hj} f(\text{net}_h^k)\right) \right\}^2 \\ &= \frac{1}{2} \sum_j \left\{ t_j^k - f\left(\sum_h w_{hj} f\left(\sum_i w_{ih} x_i^k\right)\right) \right\}^2 \end{aligned}$$

• 复合函数求导数

设 $f(x)=g_1(g_2(g_3(\cdots g_n(x))))$,

令 $h_2 = g_2(g_3(\cdots g_n(x)))$,

$h_3 = g_3(g_4(\cdots g_n(x))), \cdots$

$$\begin{aligned} & \frac{\partial f(x)}{\partial x} \\ &= \frac{\partial g_1(h_2)}{\partial h_2} \frac{\partial h_2}{\partial x} \\ &= \frac{\partial g_1(h_2)}{\partial h_2} \frac{\partial h_2(h_3)}{\partial h_3} \frac{\partial h_3}{\partial x} \\ &= \cdots \cdots \\ &= \frac{\partial g_1(h_2)}{\partial h_2} \frac{\partial h_2(h_3)}{\partial h_3} \cdots \frac{\partial g_n(x)}{\partial x} \end{aligned}$$

设 $f(x)=g_1(\textcolor{red}{g_2}(\textcolor{red}{g_3}(\cdots \textcolor{red}{g_n}(x))))+\textcolor{blue}{t_2}(\textcolor{blue}{t_3}(\cdots \textcolor{blue}{t_m}(y))))$,

令 $\textcolor{green}{H_2} = \textcolor{green}{g_2}(\textcolor{green}{g_3}(\cdots \textcolor{green}{g_n}(x)))+\textcolor{green}{t_2}(\textcolor{green}{t_3}(\cdots \textcolor{green}{t_m}(y)))$,

$h_2 = g_2(g_3(\cdots g_n(x)))$,

$h_3 = g_3(g_4(\cdots g_n(x))), \cdots$

$$\begin{aligned} \frac{\partial f(x)}{\partial x} &= \frac{\partial g_1(H_2)}{\partial H_2} \frac{\partial H_2}{\partial x} \\ &= \frac{\partial g_1(H_2)}{\partial H_2} \left(\frac{\partial h_2(h_3)}{\partial h_3} \frac{\partial h_3}{\partial x} + 0 \right) \\ &= \cdots \cdots \\ &= \frac{\partial g_1(H_2)}{\partial H_2} \frac{\partial h_2(h_3)}{\partial h_3} \cdots \frac{\partial g_n(x)}{\partial x} \end{aligned}$$

- 网络训练：隐含层—输出层

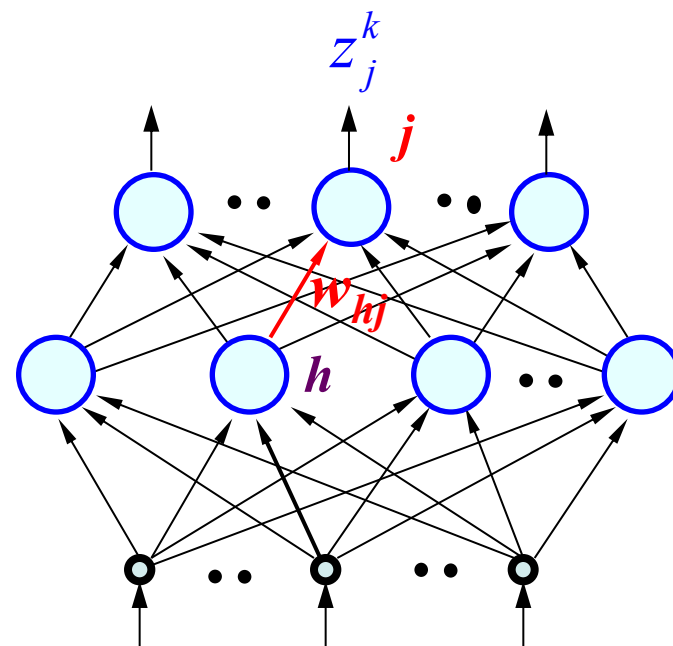
隐含层到输出层的参数更新量：

$$\begin{aligned}
 \Delta w_{hj} &= -\eta \frac{\partial E}{\partial w_{hj}} = -\eta \sum_k \frac{\partial E}{\partial net_j^k} \frac{\partial net_j^k}{\partial w_{hj}} \\
 &= \eta \sum_k (t_j^k - z_j^k) f'(net_j^k) y_h^k \\
 &= \eta \sum_k \delta_j^k y_h^k
 \end{aligned}$$

(δ规则)

$$\delta_j^k = \frac{-\partial E}{\partial net_j^k} = f'(net_j^k)(t_j^k - z_j^k) = f'(net_j^k) \Delta_j^k,$$

$$\Delta_j^k = t_j^k - z_j^k$$



- 隐含层—输出层：第 k 个训练样本对权重 w_{hj} 更新的贡献

$h \rightarrow j$, for sample k :

δ 规则:

$$\Delta w_{hj} \mid_{\text{sample } k} = \eta \delta_j^k y_h^k$$

权重所联边的起始结点（隐含结点 h ）的输出

权重所联边的指向结点（输出结点 j ）的误差

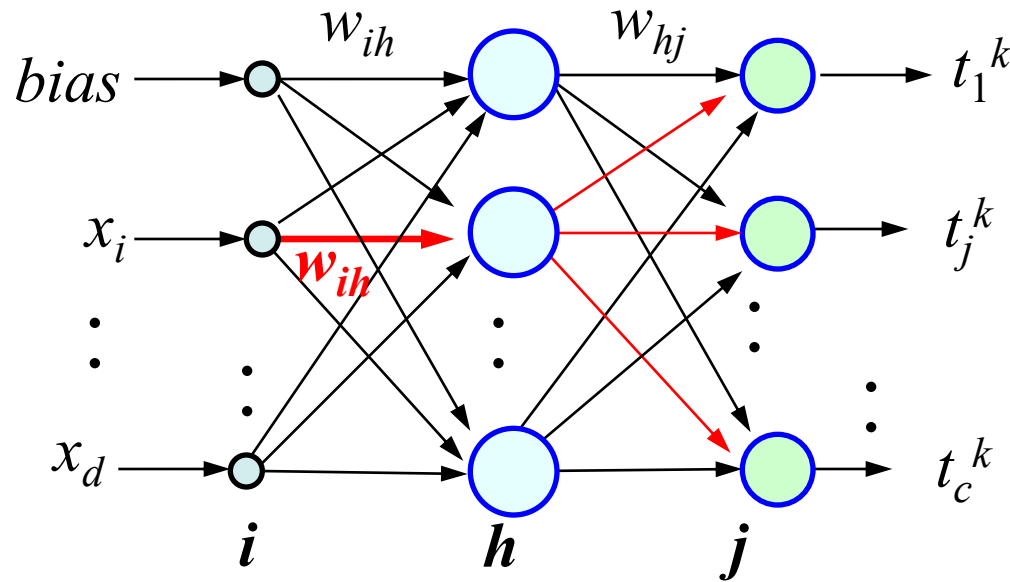
$$\delta_j^k = f'(net_j^k) \Delta_j^k, \quad \Delta_j^k = t_j^k - z_j^k$$

误差在权重所联边的指向结点处计算。误差大小等于该结点收集到的误差乘以激励函数对“该结点加权和”的导数。

对于输入层到隐含层结点连接的边的权重更新量 Δw_{ih} ，必须考虑将 $E(\mathbf{w})$ 对 w_{ih} 求导，需利用**分层链路法**。

输入层至隐含层权重更新：

$$E(\mathbf{w}) = \frac{1}{2} \sum_{k,j} (t_j^k - z_j^k)^2 = \frac{1}{2} \sum_{k,j} \left\{ t_j^k - f \left(\sum_h w_{hj} f \left(\sum_i w_{ih} x_i^k \right) \right) \right\}^2$$

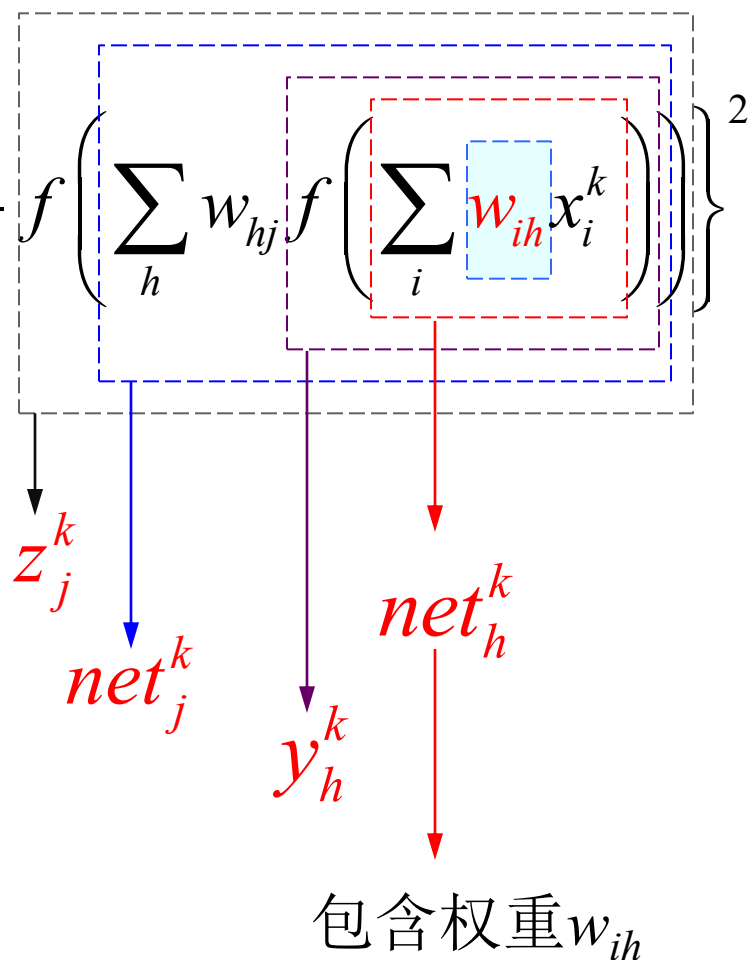


样本： $x_i^k \rightarrow \text{net}_h^k \rightarrow y_h^k \rightarrow \text{net}_j^k \rightarrow z_j^k \quad t_j^k$

输入层至隐含层权重更新：

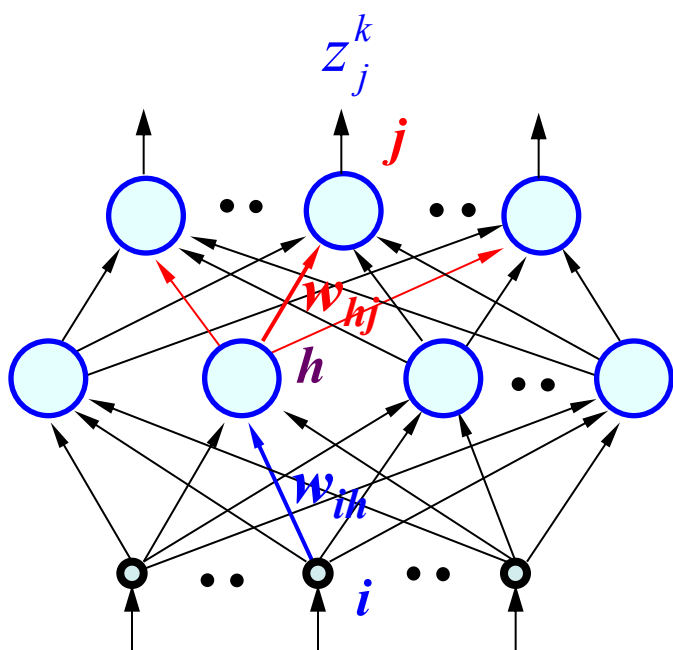
$$E(\mathbf{w}) = \frac{1}{2} \sum_{k,j} (t_j^k - z_j^k)^2 = \frac{1}{2} \sum_{k,j} \left\{ t_j^k - f \left(\sum_h w_{hj} f \left(\sum_i w_{ih} x_i^k \right) \right) \right\}^2$$

$$\begin{aligned} net_h^k &= \sum_i w_{ih} x_i^k, & y_h^k &= f(net_h^k) \\ net_j^k &= \sum_h w_{hj} y_h^k, & z_j^k &= f(net_j^k) \end{aligned}$$



待更新权重 w_{ih} 的增量:

$$\Delta w_{ih} = -\eta \frac{\partial E}{\partial w_{ih}} = -\eta \sum_{k,j} \frac{\partial E}{\partial z_j^k} \frac{\partial z_j^k}{\partial w_{ih}} \quad \text{(链式法则)}$$



$$\begin{aligned} &= -\eta \sum_{k,j} \frac{\partial E}{\partial z_j^k} \frac{\partial z_j^k}{\partial net_j^k} \frac{\partial net_j^k}{\partial w_{ih}} \\ &= -\eta \sum_{k,j} \frac{\partial E}{\partial z_j^k} \frac{\partial z_j^k}{\partial net_j^k} \frac{\partial net_j^k}{\partial y_h^k} \frac{\partial y_h^k}{\partial w_{ih}} \\ &= -\eta \sum_{k,j} \frac{\partial E}{\partial z_j^k} \frac{\partial z_j^k}{\partial net_j^k} \frac{\partial net_j^k}{\partial y_h^k} \frac{\partial y_h^k}{\partial net_h^k} \frac{\partial net_h^k}{\partial w_{ih}} \\ &= -\eta \sum_k \left[\sum_j \frac{\partial E}{\partial z_j^k} \frac{\partial z_j^k}{\partial net_j^k} \frac{\partial net_j^k}{\partial y_h^k} \frac{\partial y_h^k}{\partial net_h^k} \right] \frac{\partial net_h^k}{\partial w_{ih}} \\ &= -\eta \sum_k \frac{\partial E}{\partial net_h^k} \frac{\partial net_h^k}{\partial w_{ih}} = -\eta \sum_k \frac{\partial E}{\partial net_h^k} x_i^k \end{aligned}$$

输入-隐层：第 k 个训练样本对权重 w_{ih} 的贡献

$i \rightarrow h$, for sample k :

δ 规则:

$$\Delta w_{ih} \mid_{\text{sample } k} = \eta \delta_h^k x_i^k$$

w_{ih} 所连接的边的起始结点（输入层结点 i ）的输出（此时即为样本第 i 个分量）

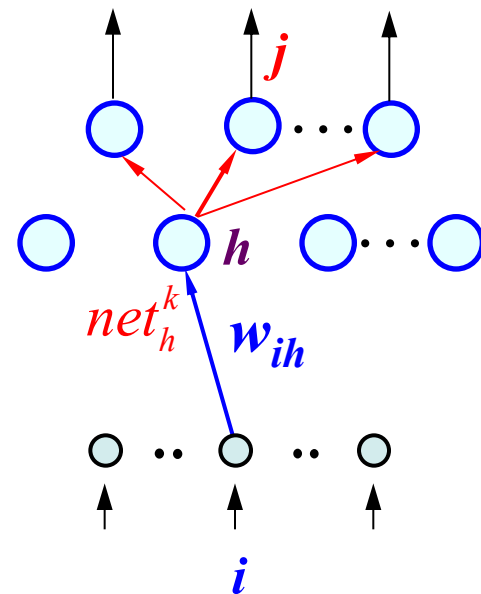
$$\delta_h^k = -\frac{\partial E}{\partial \text{net}_h^k}$$

w_{ih} 所连接的边的指向结点（隐含结点 h ）的误差

误差信号: $\delta_h^k = \frac{-\partial E}{\partial net_h^k}$

$$\begin{aligned}
 &= \sum_j \frac{-\partial E}{\partial net_j^k} \frac{\partial net_j^k}{\partial y_h^k} \frac{\partial y_h^k}{\partial net_h^k} \\
 &= \frac{\partial y_h^k}{\partial net_h^k} \sum_j \frac{-\partial E}{\partial net_j^k} \frac{\partial net_j^k}{\partial y_h^k} \\
 &= f'(net_h^k) \sum_j w_{hj} \frac{-\partial E}{\partial net_j^k} \\
 &= f'(net_h^k) \sum_j w_{hj} \delta_j^k
 \end{aligned}$$

这就是误差反传！



输入-隐层：第 k 个训练样本对权重 w_{ih} 更新的贡献

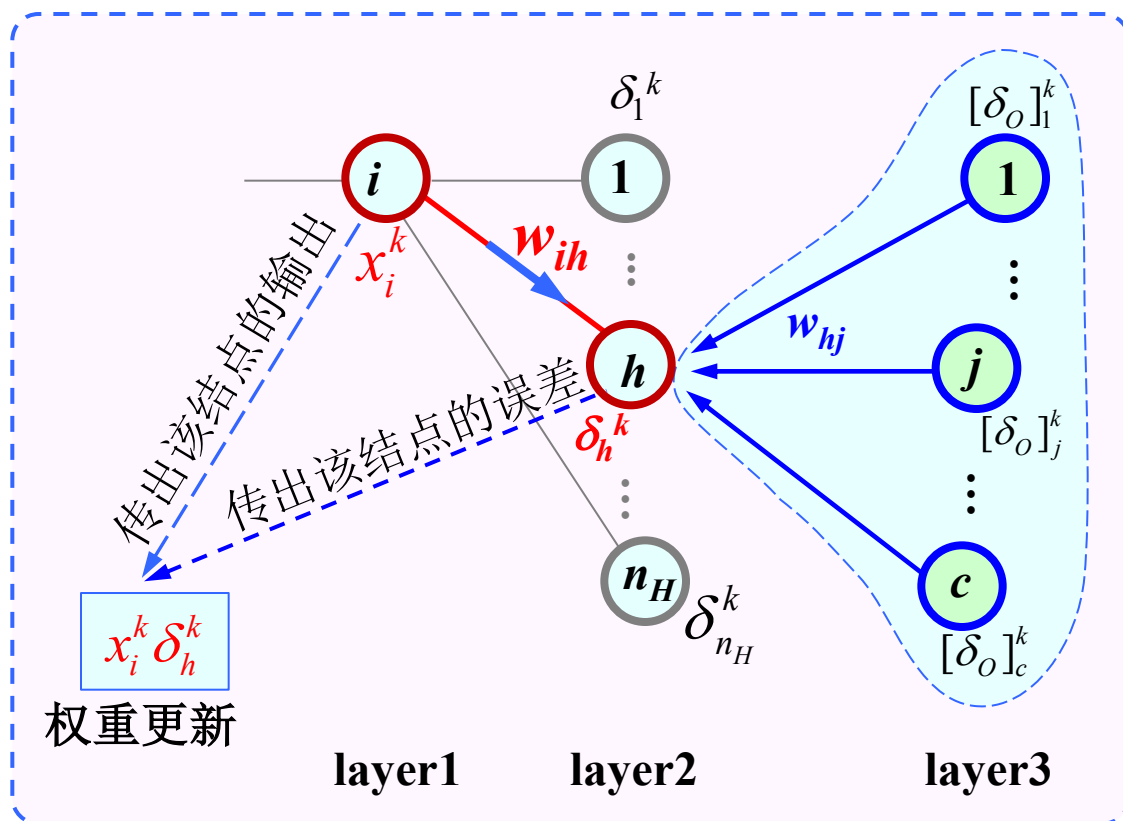
$$\delta_h^k = \frac{-\partial E}{\partial net_h^k} = f'(net_h^k) \sum_j w_{hj} \delta_j^k$$

误差大小等于从前一层收集误差的加权和乘以激励函数对“结点净输入”的导数。

误差在权重所联边的指向结点处计算。

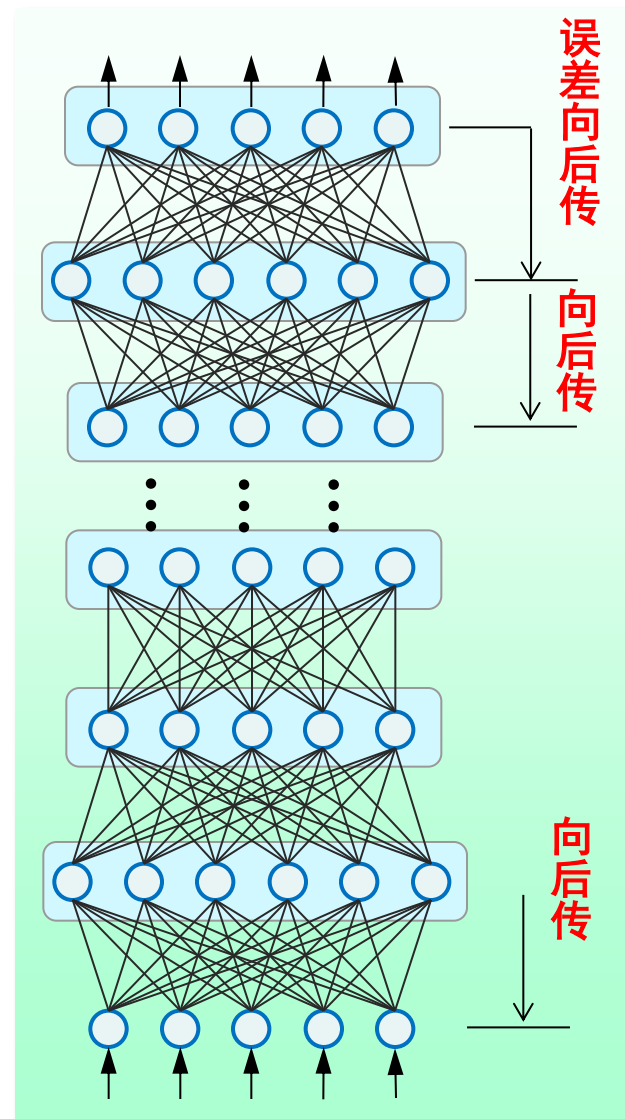
6.4.2 BP算法

- w_{ih} 权重更新



w_{ih} 的更新量:

$$\Delta w_{ih} = \eta x_i^k \delta_h^k, \delta_h^k = f'(net_h^k) \sum_j w_{hj} [\delta_o]_j^k$$



6.4.2 BP算法

- 网络训练

- BP算法对任意层的权重更新量的一般形式：

$$\Delta w_{h \rightarrow h'} = \eta \sum_k \delta_{h'} y_h^k$$

- 单个训练样本的贡献：

$$\Delta w_{h \rightarrow h'} = \eta \cdot \delta_{h'} \cdot y_h$$

$$\delta_{h'} = f'(net_{h'}) \sum_{h''} w_{h' \rightarrow h''} \delta_{h''}$$

$$y_h = f\left(\sum_{h^*} w_{h^* \rightarrow h} y_{h^*}\right)$$

从后一层各结点
收集误差

从前一层各结点
收集输出

下标 h 和 h' 分别指“待更新权重”所连边的起始结点和指向结点， y_h 代表起始结点的输出， $\delta_{h'}$ 表示指向结点的误差。

6.4.2 BP算法

- 随机单样本更新

Stochastic Backpropagation

- 1 begin initialize: n_H , $\mathbf{w}$, η , criterion θ , $k=0$
 - 2 do $k \leftarrow k + 1 \pmod n$
 - 3 randomly chose a sample $\mathbf{x}^k$
 - 4 $w_{hj} \leftarrow w_{hj} + \eta \delta_j^k y_h^k, \quad w_{ih} \leftarrow w_{ih} + \eta \delta_h^k x_i^k$
 - 5 until $\| \nabla E(\mathbf{w}) \| < \theta$
 - 6 return $\mathbf{w}$
 - 7 end
-

适用于大规模数据

- 批量更新算法

Batch Backpropagation

1 begin initialize: n_H , $\mathbf{w}$, η , criterion θ , $r=0$

2 do $r \leftarrow r + 1$ (increment epoch)

3 $k = 0$, $\Delta w_{ih} = 0$, $\Delta w_{hj} = 0$

4 do $k \leftarrow k+1$

5 select the sample $\mathbf{x}^k$

6 $\Delta w_{hj} \leftarrow \Delta w_{hj} + \eta \delta_j^k y_h^k$, $\Delta w_{ih} \leftarrow \Delta w_{ih} + \eta \delta_h^k x_i^k$

7 until $k = n$

8 $w_{hj} \leftarrow w_{hj} + \Delta w_{hj}$, $w_{ih} \leftarrow w_{ih} + \Delta w_{ih}$

// 所有样本完成之后再更新

9 until $\| \nabla E(\mathbf{w}) \| < \theta$

10 return $\mathbf{w}$

11 end

下次课内容

- 多层前向网络
 - BP算法讨论
- 径向基函数网络
- 反馈神经网络基础
- 自组织映射
- 深度学习综述

致谢

- PPT由向世明老师提供

Thank All of You!
(Questions?)

张燕明

ymzhang@nlpr.ia.ac.cn

people.ucas.ac.cn/~ymzhang

模式分析与学习课题组 (PAL)

中科院自动化研究所·多模态人工智能系统实验室